

Validación de Entradas en el Formulario de Tareas

Duración estimada: 2 horas

● OBJETIVO DE LA TAREA

Implementarás las primeras interacciones de tu aplicación utilizando JavaScript.

Deberás validar los datos ingresados en el formulario de creación de tareas antes de permitir que sean procesados.

La validación debe identificar entradas inválidas y proporcionar retroalimentación clara al usuario.

Resultado esperado

Al finalizar esta actividad tu formulario deberá:

- ✓ Validar que los campos requeridos no estén vacíos.
- ✓ Mostrar un mensaje de error claro cuando los datos sean inválidos.
- ✓ Ocultar el mensaje de error cuando la información sea correcta.

ESTA ACTIVIDAD CIERRA EL SPRINT 1

Además de implementar la validación, deberás dejar tu repositorio organizado y actualizado, ya que al finalizar se realizará una revisión integral de las tres tareas del Sprint.

Desarrollo de la actividad

1 Paso 1. Agregar o verificar el formulario de tareas

Verifica que tu `index.html` cuente con un formulario que permita capturar la información de una tarea, con los siguientes campos:

- Nombre
- Descripción
- Fecha de entrega
- Estado
- Un botón para enviar el formulario

Si ya construiste este formulario en tareas anteriores, revisalo y ajústalo; evita duplicar código innecesariamente y trabaja sobre el proyecto existente.

2 Paso 2. Implementar la validación con JavaScript

2.1 Crear el archivo JavaScript

Crea un archivo llamado `app.js` e inclúyelo correctamente dentro de `index.html`.

2.2 Crear la función de validación

Implementa una función llamada:

```
validFormFieldInput(data)
```

Esta función será responsable de revisar los datos del formulario y determinar si son válidos antes de continuar con el procesamiento de la tarea.

2.3 Identificar los campos del formulario

Cada campo del formulario debe tener un atributo `id` que permita identificarlo desde JavaScript.

Ejemplo:

```
const new-task-name-input = document.querySelector('#newTaskNameInput');
const name = newTaskNameInput.value;
```

- `querySelector()` permite seleccionar un elemento del HTML.
- `.value` permite obtener el valor ingresado por el usuario.

2.4 Comprobar los datos

Utiliza `console.log()` para comprobar que los datos ingresados se están recuperando correctamente antes de avanzar con la validación.

Ejemplo:

```
const new-task-name-input = document.querySelector('#newTaskNameInput');
const name = newTaskNameInput.value;

console.log("name: " + name);
```

Tabla de validaciones

Como mínimo, cada campo debe cumplir con la siguiente condición:

Campo	Validación
Nombre	No puede estar vacío
Descripción	No puede estar vacía
Fecha de entrega	Debe contener una fecha
Estado	Debe tener una opción seleccionada

3

Paso 3. Mostrar errores al usuario

Implementa retroalimentación visual utilizando el componente **Alert** de Bootstrap o **SweetAlert2**:

- Agrega un componente de alerta dentro de la interfaz.
- Mantén el mensaje de error oculto inicialmente.
- Muestra el mensaje cuando el usuario envíe información inválida.
- Oculta nuevamente el mensaje cuando los datos sean válidos.

Lógica de la validación

Usuario envía formulario → JavaScript revisa los datos → si hay errores, se muestra la alerta → si los datos son válidos, se oculta la alerta.

Prueba tu código

Prueba diferentes escenarios; no te limites solo al caso exitoso.

Caso 1 — Formulario vacío

Presiona el botón de enviar sin ingresar información.

Resultado esperado: debe mostrarse el mensaje de error.

Caso 2 — Campos incompletos

Completa solo algunos campos y envía el formulario.

Resultado esperado: la aplicación identifica los datos faltantes y muestra el mensaje correspondiente.

Caso 3 — Formulario válido

Completa todos los campos correctamente.

Resultado esperado: no debe mostrarse el mensaje de error.

Cierre del Sprint 1

IMPORTANTE

Esta tarea no representa únicamente el cierre de la Tarea 3, sino también el cierre del Sprint 1 completo.

Al finalizar, revisa que tu proyecto tenga todos los avances de las tres tareas: se realizará una revisión integral del Sprint.

Entrega final del Sprint 1

Deberás entregar el link de tu repositorio de GitHub, el cual debe contener los avances correspondientes a:

Tarea 1

Tarea 2

Tarea 3

La revisión considerará la evolución del proyecto y la trazabilidad de los cambios realizados.

HISTORIAL DE COMMITS

El repositorio será revisado considerando el historial de commits: se verificará la fecha de cada uno para comprobar la evolución del proyecto durante el Sprint.

No presentes únicamente un proyecto finalizado sin historial de trabajo; la intención es evidenciar tu proceso de desarrollo.

README del repositorio

Como parte del cierre del Sprint 1, crea o actualiza el archivo `README.md` de tu repositorio con la siguiente información mínima:

1 Descripción del proyecto

Una breve explicación del Planificador de Tareas.

2 Trello

Enlace al tablero utilizado para organizar el Sprint 1.

3 Figma

Enlace al archivo con el wireframe del proyecto.

4 Demo

Enlace al proyecto publicado mediante GitHub Pages.

Publicar el proyecto en GitHub Pages

Guía rápida de consulta.

1. Ingresa al repositorio en GitHub.
2. Ve a **Settings**.
3. Selecciona **Pages**.
4. En **Build and deployment**, selecciona la fuente correspondiente al repositorio.
5. Selecciona la rama `main`.
6. Selecciona la carpeta `/root`.
7. Guarda la configuración.
8. Espera la publicación.
9. Copia el enlace generado por GitHub Pages.
10. Agrega ese enlace a tu `README.md`.

El enlace de GitHub Pages será utilizado como la versión publicada del proyecto para la revisión del Sprint.

Checklist de cierre del Sprint

Verifica cada punto antes de entregar tu proyecto.

Proyecto

- ☐ Tarea 1 implementada.
- ☐ Tarea 2 implementada.
- ☐ Tarea 3 implementada.

JavaScript

- ☐ `.js` vinculado correctamente.
- ☐ `validFormFieldInput(data)` implementada.
- ☐ Campos recuperados correctamente mediante JavaScript.
- ☐ Validaciones funcionan correctamente.
- ☐ Mensajes de error se muestran y ocultan correctamente.

GitHub

- ☐ Repositorio actualizado.
- ☐ Commits evidencian el progreso del proyecto.
- ☐ Existe un archivo README.md.

README y Deploy

- ☐ Incluye descripción, Trello, Figma y demo.
- ☐ Proyecto publicado en GitHub Pages.
- ☐ El enlace funciona correctamente.

● RESULTADO ESPERADO AL CIERRE DEL SPRINT 1

Habrás pasado de una idea y un wireframe a una primera versión funcional de la interfaz del Planificador de Tareas, con:

- ☒ Interfaz construida con HTML y Bootstrap, tarjetas de tareas y lista de tareas.
- ☒ Formulario de creación de tareas con primeras validaciones en JavaScript.
- ☒ Repositorio organizado en GitHub, publicado en GitHub Pages y con README completo.

Este es solo el primer avance de un proyecto que seguirá creciendo durante los siguientes sprints.